

Addressables 위의 수명 스코프 리팩터

"안전한 쪽 = Global"로 도피하던 리소스 수명을, refCount 레지스트리 + 수명 스코프로 재구축해 상주 메모리를 회수하고 — 개발 빌드로 회수를 실측했다.

리소스의 실제 수명은 5종 이상인데 표현 수단이 **2개(Global/Scene)**뿐이라, 애매한 것이 전부 "안전한 쪽 = Global"로 도피해 영구 상주했다. 이를 **(key, type) refCount 레지스트리 + 수명 스코프**로 재구축해 상주 메모리를 회수하고, **수동 Release 코드는 한 줄도 추가하지 않았다**. 이 문서는 문제 → 설계 → 여정 → 실측 → 방법론을 처음부터 정리한다.

1 문제 — 2버킷의 구조적 결함

"버그"가 아니라 "표현력 부족" — 개발자를 Global 도피로 몰아넣는 구조였다.

초기 리소스 관리는 **Global(영구) / Scene(썬)** 두 버킷뿐이었다. 그런데 실제 수명은 훨씬 다양하다 — **팝업(스택 있는 동안)·파티(썬 넘어 생존)·던전(던전 동안)·영구**. 표현할 계층이 없으니, "언제 해제해야 할지 애매한 것"은 전부 **안전한 쪽(Global)**으로 도피했고, 한 번 Global에 박힌 것은 게임 종료까지 안 내려왔다.

- **중복·비대칭** — 같은 UI 기능인데 본체는 Global, 서브아이템은 Scene으로 수명이 갈라짐.
- **수동 Release의 함정** — Addressables는 핸들 단위 수동 짝맞추기라, 호출부가 Release를 "기억"해야 하고 그 기억의 실패가 곧 누수였다.

2 Before — 문제를 숫자로 (Baseline)

"애매하면 Global"의 비용을 MB로 환산. 개선의 근거를 먼저 측정으로 확보.

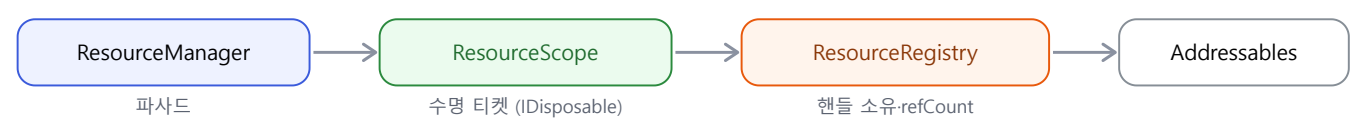
증상	측정치 (Before)
팝업 한 세션 열람 = 영구 고정	global 핸들 50→58→61 단조 증가 / 메모리 +287MB 영구 고정 (닫아도.썬 옮겨도 안 내려옴)
핸들 메모리의 95%가 Global	1287MB(Global) vs 65MB(Scene) — "썬 버킷은 정상, 문제는 스코프"가 수치로 확인
아틀라스 캐시 폭증	루트 UI 1회 노출에 AtlasSpriteCache 36→220 , 장시간 ~300 포화 + 해제 경로 없음
몬스터 HP바 누수	DDOL 캔버스에 방치돼 자가정리 안 됨 → Memory Profiler GameObject +291

진단

썬 버킷의 썬 단위 해제는 **정상**. 진짜 문제는 "**썬보다 짧은 수명(팝업·던전)**"과 "**썬보다 긴 수명(파티)**"을 표현할 계층이 없다는 것. → 스코프 도입의 근거가 수치로 확보됨.

3 설계 — 2버킷 → 레지스트리 + 스코프

"누가 Release하나"라는 기억의 문제를, 수명 단위 자동 정산이라는 구조로 대체.



설계 3원칙

- ① 소유는 한 곳

핸들 소유는 레지스트리 한 곳, 스코프는 티켓(`HashSet<ResourceKey>`)만 쥔다
→ "누가 Release하나" 문제 소멸.
- ② refCount = 스코프 수

스코프 내 중복 로드는 `HashSet.Add` 반환값으로 refCount 1. refCount = "이 핸들을 소유한 서로 다른 스코프 수".
- ③ 취소 ≠ 해제

토큰은 기다림 중단만. 수명은 스코프 소유가 결정 → use-after-release가 구조적으로 불가.

수명 계층: Global (영구) · Scene (씬) · Party (파티 유지, 씬 전환 생존) · Popup (스택 있는 동안). — **Dungeon 스코프는 측정상 이미 해결돼 만들지 않았다(\$7 근거 기반 축소).**

4 여정 — Phase 0 → 4 + 후속

"재배치가 목적, 스코프는 수단" — 수치는 전부 수명 재배치(Phase 3)가 움직인다.

단계	한 일	성과 / 검증
Phase 0	Baseline 측정 + 계측 도구 (<code>ResourceMetrics</code> , 디버그 창)	P1 정량 확인, 측정 시나리오 고정
Phase 0.5	아틀라스 콘텐츠 트랙 — Standing/EscapeMenu 해체(전량→사용량 비 례 로드)	UI_Info 히치 219.7→123.4ms, 1명=1장 로드. 측정 도구 ×6.8 오차 발견·보정
Phase 2	레지스트리+스코프 코어 신설, 파사드·씬 흐 름 이관	씬 왕복 잔존 = Baseline과 동일(수치 불변 = 구조 교체 성공)
Phase 3a~d	수명 재배치 — 팝업→Scene, 캐릭→Party, Global 라벨 42→12, 아틀라스 캐시 수명화	진입 Global -60%, 합계 -41%, 던전 왕복 +18.7→0
Phase 4	풀↔에셋 수명 통합(refCount 티켓) + <code>@Pool_Root</code> DDOL 제거	왕복 flat 358.6→359.1MB, 순서 의존→구조 보 장
후속	R4(NoCache 값복사) · HUD 아틀라스 Party 재배치(churn 제거) · 셀프 코드리뷰 2건	재로드 3→0, 취소 유령 핸들 + 아틀라스 중복 전 개 자가 발견·수정

5 헤드라인 숫자

진입 직후, Baseline v2 → 완료. 절대값이 아니라 증감·비율(신뢰도 "강").

-60%

Global 핸들 메모리
513MB → 205MB

-41%

진입 전체 합계
617MB → 363MB

-72%

Global 단조 증가
+56MB → +16MB

≈0

던전 왕복 영구 증가
+18.7MB → 해소

대가(정직하게): 진입 -308MB ↔ 팝업 첫 오픈 +14~20ms. **상시 부담을 일회성 지연으로 바꾼 거래.** 재오픈은 회귀 없음(~9ms). 풀↔에셋 수명은 순서 의존(핑크 위험) → **왕복 flat**으로 구조 보장.

6 개발 빌드 실측 (2026-08-11) — 회수를 눈으로

에디터 x2 텍스처 CPU 사본이 없는 런타임 총량. "개발 빌드 실측 0회" 각주 해소.

체크포인트	총 할당	텍스처	그래픽스 드라이버
① 진입 직후	332.9	140.7	471.2
② 팝업 5종 첫 오픈 후	337.1	168.8 (+28.1)	505.6
③ 재오픈 후	337.1	168.8 (무비용)	505.3
④ 던전 왕복 후	343.9	142.1 (-26.7, 진입 복귀)	582.9
⑤ 종료	344.1	142.1	582.9

머니샷 — 텍스처 회수

텍스처 메모리가 진입 140.7 → 팝업 세션 168.8(+28) → 던전 왕복 후 142.1(진입 수준 복귀). 팝업이 씬 스코프에 적재됐다가 씬 회전에서 **refCount 자동 정산으로 -26.7MB 반납** — 수동 Release 0줄, 실제 빌드에서. 재오픈은 텍스처·총할당 완전 동일(UI_Info 첫 23.1ms → 재 2.3ms) = 캐시 히트 무비용.

정직한 각주: 그래픽스 드라이버 메모리(471→583)는 안 내려옴 = 드라이버 지연 반납 특성이라 **회수 판정** 주 신호는 텍스처. 이 [ResourceMemory] 는 프로세스 총 할당이고 \$5의 513MB는 버킷별 핸들 메모리라 다른 지표(직접 차감 금지). UI_Info 첫오픈 334ms는 알려진 UI_Info 구조 문제(n=1).

7 방법론 — 결과물보다 "어떻게 판단했나"

이 리팩터의 진짜 값은 감축량이 아니라, 측정으로 판단하는 규율이다.

- "측정 → 진단 → 구조 → 검증" 루프 — 각 단계가 자기 몫만 정리하고 나머지를 다음 단계에 정확히 넘김 (잔여 증가가 3b/3d로 깔끔히 분해).
- 숫자를 믿기 전에 도구를 검증 — 측정 도구가 아틀라스를 x6.8 과대 계상하던 계통 오차를 플레이 모드 교차 검증으로 발견·보정. 계측 이관 후엔 isGlobal 한 줄만 토글해 단일 변수 A/B로 도구 등가성까지 실측 (512.8MB 소수점 일치).
- 계측이 기존 버그 3건을 드러냄 — 전부 뿌리가 "에셋 수명 ≠ 인스턴스 수명". "메모리가 줄었다"가 실은 누수를 못 센 결과일 수 있음을 R6에서 실증(합계 보존 = 유령이 회계에 드러남 = 성과 아닌 비용으로 정직 서술).
- 근거 기반 축소 — Dungeon 스코프·프레임 예산 스트리밍·UI 루트 DDOL 대공사를 측정 근거로 제외. "명세를 따르지 않고 측정을 따랐다"(던전 왕복이 이미 +0.9MB).
- 셀프 코드리뷰 — 취소 경로 유령 핸들 누수 + 아틀라스 중복 전개를 자가 발견·수정, 실측으로 누수 0·왕복 flat 재검증.

8 면접 방어 질문 3종

질문	답
"Addressables가 이미 refCount인데 왜 위에 또?"	Addressables refCount는 핸들 단위 수동 짝맞추기 — 호출부가 Release를 '기억'해야 하고, 그 기억의 실패가 HP바 누수였다. 스코프는 기억을 구조(수명 단위 자동 정산)로 대체 한 것.
"본인이 만든 문제를 본인이 고친 것 아닌가?"	맞다, 숨기지 않는다. 2버킷은 초기 설계였고 계측으로 한계를 발견 → 개인의 주의력이 아니라 구조로 재발 차단 . 이 프레이밍이 성장 서사.
"1인 프로젝트에 오버엔지니어링 아닌가?"	팝업 세션 +287MB라는 측정치가 필요성을 증명. 반대로 던전 스코프·프레임 예산·DDOL 대공사는 측정 근거로 제외 = 필요한 것만 했다는 증거.

9 한계 (반드시 동반)

- 버킷별 **절대값**은 에디터 측정(CPU 사본 과대 계상) → 증감·비율만 인용. 단 **총량 절대값은 개발 빌드 실측 확보(\$6)**, `Use Existing Build` = 실제 번들 로 증감·비율은 신뢰도 "강".
- 그래픽스 드라이버 메모리는 지연 반납 특성 → 회수 판정 주 신호는 **텍스처**.
- 히치 ms는 $n=1\sim3$, **15ms 이하 차이 주장 금지**. TMP 동적 폰트가 세션마다 절대값을 흔드니 버킷 합계를 주 지표로.

Operation Kivotos · ResourceSystem · Portfolio (Full Narrative) · 2026-08-11 · 근거: Baseline.md(\$0~\$13) · Phase3_Result.md · Phase4_Result.md · ProblemSolving_Log.md · Portfolio_Summary.md · 빌드 실측 metrics_20260811_004330.log · 코어: Assets/Scripts/Managers/Core/Resource/{ResourceManager,ResourceRegistry,ResourceScope,ResourceMetrics}.cs